

GLOBALRAIN

Practices for Secure Software Report

Table of Contents

PRACTICES FOR SECURE SOFTWARE REPORT 1
DOCUMENT REVISION HISTORY 3
CLIENT 3
DEVELOPER 4
1. ALGORITHM CIPHER 4
2. CERTIFICATE GENERATION 7
3. DEPLOY CIPHER 7
4. SECURE COMMUNICATIONS 7
5. SECONDARY TESTING 8
6. FUNCTIONAL TESTING 8
7. SUMMARY 9
8. INDUSTRY STANDARD BEST PRACTICES 10

Document Revision History

Version	Date	Author	Comments
1.0	10/17/2026	Jonathan Miranda	Initial secure software report

Client



Developer

Jonathan Miranda

1. Algorithm Cipher

I would like to recommend the use of the Advanced Encryption Standard (AES) with a 256-bit secret key in Galois/Counter Mode (AES-256-GCM) to protect the data of users of the Artemis Financial application. The information that users of the Artemis Financial application will send to and receive from the application can be protected using symmetric encryption, which uses the same secret key for encryption and decryption to make the information unreadable by third parties. AES processes information in 128-bit blocks and supports key lengths of 128, 192, and 256 bits. I recommend AES-256-GCM because it is a strong and widely used mode of operation for the Advanced Encryption Standard that also provides authentication to detect unauthorized changes to encrypted data (National Institute of Standards and Technology [NIST], 2007, 2023). This is an appropriate choice for encrypting information sent through the HTTPS protocol, although I did not verify the specific TLS cipher suite used during testing.

To provide checksum verification and ensure data integrity in the refactored application, I used the Secure Hash Algorithm 256 (SHA-256) as described in FIPS 180-4: Secure Hash Standard (NIST, 2015). SHA-256 belongs to the SHA-2 family of hashing algorithms and produces a 256-bit hash value that is represented by 64 hexadecimal characters. Unlike encryption, hashing is a one-way operation, and therefore data that has been hashed cannot be directly reconstructed from the resulting hash values. However, by calculating a hash value for data and comparing the

resulting value against a trusted checksum for that same data, unexpected changes can be identified. Hashing alone does not verify where the data originated. In the refactored code, I used Java's MessageDigest class to create a SHA-256 checksum from a unique string which contained my name. To provide independent verification of the checksums created by the Java code, I used PowerShell to calculate the same SHA-256 checksum. The results of the two independent calculations are shown in Section 3 of this report. As can be seen from Section 3, both methods created identical results for identical input data.

One type of encryption used in cryptography is symmetric encryption, which uses a shared secret key to encrypt and decrypt information. Another type of encryption is asymmetric encryption, which uses pairs of keys. Public keys can be used to encrypt information that can only be decrypted using the corresponding private key. Private keys can also be used to create digital signatures that are verified using public keys. I used a 3072-bit RSA key pair with the SHA256withRSA signature algorithm to create a self-signed certificate for Artemis Financial. The private key for the certificate was stored in a password-protected PKCS12 keystore, and the public certificate with its corresponding public key was exported as a CER file. The server identity of Artemis Financial can then be authenticated during the TLS handshake provided the client already trusts the certificate. Cryptographically secure random numbers are critical to generating secret keys and establishing secure TLS sessions because predictable values can weaken encryption. SHA-256 checksums, however, are completely deterministic and do not require random numbers.

Encryption is constantly evolving. The Data Encryption Standard (DES) was introduced in 1977 and was secured with a 56-bit key. Unfortunately, with the rise in computing power, it became possible to complete brute-force attacks against DES-encrypted data. The Advanced Encryption Standard (AES) was standardized in 2001 to combat this problem. AES supports 128-bit, 192-bit, and 256-bit keys, all of which provide strong protection when implemented correctly. In a similar manner, secure hash functions have also evolved as problems were found in older algorithms such as MD5 and SHA-1. These are no longer considered suitable for security-sensitive applications that require collision resistance. Current encryption and hashing algorithms, in conjunction with secure TLS communication, provide stronger confidentiality and data integrity protection for sensitive information (NIST, 2015, 2023).

The refactored application has additional security features. The application generates SHA-256 checksums to support data integrity verification. Data transferred between Artemis Financial and its users is encrypted through HTTPS. I tested the additional security features on my local machine. For testing the SHA-256 checksums as well as the HTTPS feature of Artemis Financial, I created a self-signed certificate. I modified the Artemis Financial application to listen on port 8443 for HTTPS traffic. The same checksum was produced for a unique string containing my name by independently calculating the checksum using PowerShell and the refactored Java application. The additional security features were successfully integrated into the application, which still works as expected. However, for production use, Artemis Financial will need a certificate issued by a trusted Certificate Authority (CA) and must safeguard the certificate and its corresponding private key. Furthermore, several vulnerable dependencies identified by OWASP Dependency-Check still need to be reviewed and fixed in addition to the security features already implemented.

2. Certificate Generation

```
PS C:\Users\jmira\OneDrive\Documents\SNHU\CS-305\w7\ssl-server_student> keytool -printcert -file "artemis-certificate.cer"
Owner: CN=localhost, OU=CS305, O=Artemis Financial, C=US
Issuer: CN=localhost, OU=CS305, O=Artemis Financial, C=US
Serial number: 73475709
Valid from: Sat Oct 17 13:38:30 EDT 2026 until: Sun Oct 17 13:38:30 EDT 2027
Certificate fingerprints:
         SHA1: E0:3F:58:8F:89:E7:33:2D:FD:82:1A:A9:23:7F:59:2E:2A:4F:E2:49
         SHA256: 08:74:7D:12:1E:BF:F2:66:50:FE:86:14:E8:1B:C5:DE:59:7B:65:64:68:2E:4B:0E:A7:93:F9:72:E2:04:13:AE
Signature algorithm name: SHA256withRSA
Subject Public Key Algorithm: 3072-bit RSA key
Version: 3

Extensions:

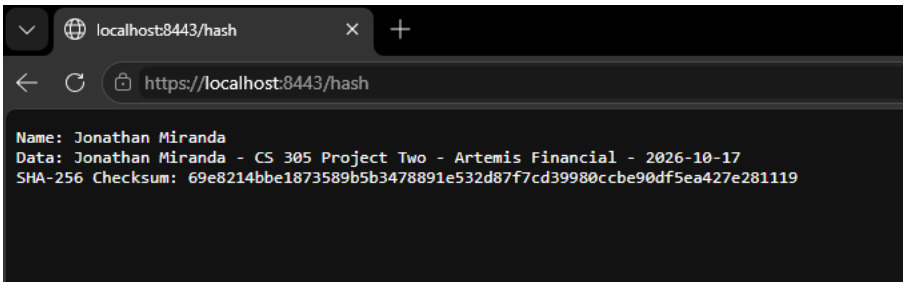
#1: ObjectId: 2.5.29.17 Criticality=false
SubjectAlternativeName [
  DNSName: localhost
  IPAddress: 127.0.0.1
]

#2: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 62 72 65 8A 1C C2 85 34   D8 9D 33 32 2F 1A 97 EE  bre....4..32/...
0010: C3 43 58 35                                        .CX5
]
]
```

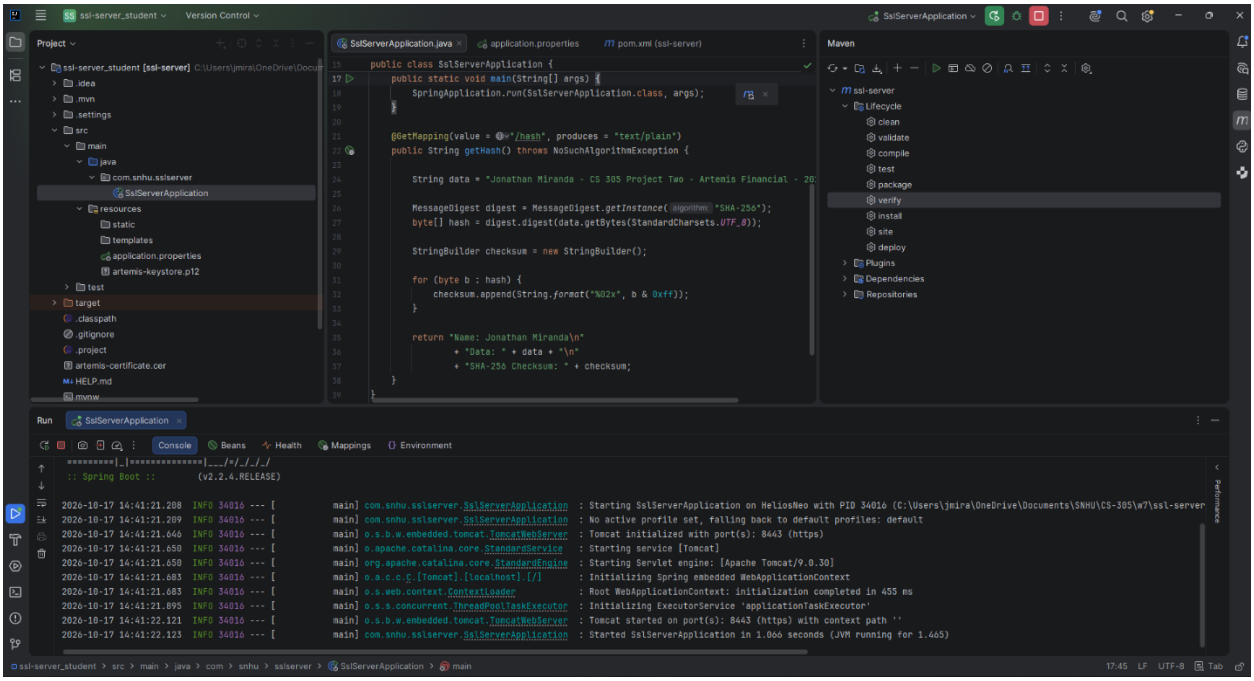
3. Deploy Cipher

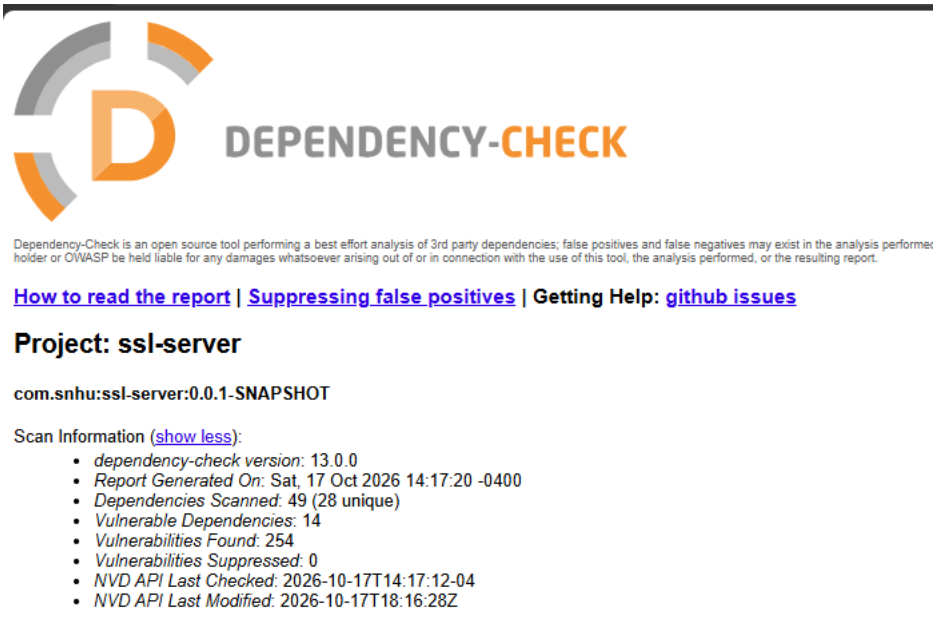
```
Name: Jonathan Miranda
Data: Jonathan Miranda - CS 305 Project Two - Artemis Financial - 2026-10-17
SHA-256 Checksum: 69e8214bbe1873589b5b3478891e532d87f7cd39980ccbe90df5ea427e281119
```

4. Secure Communications

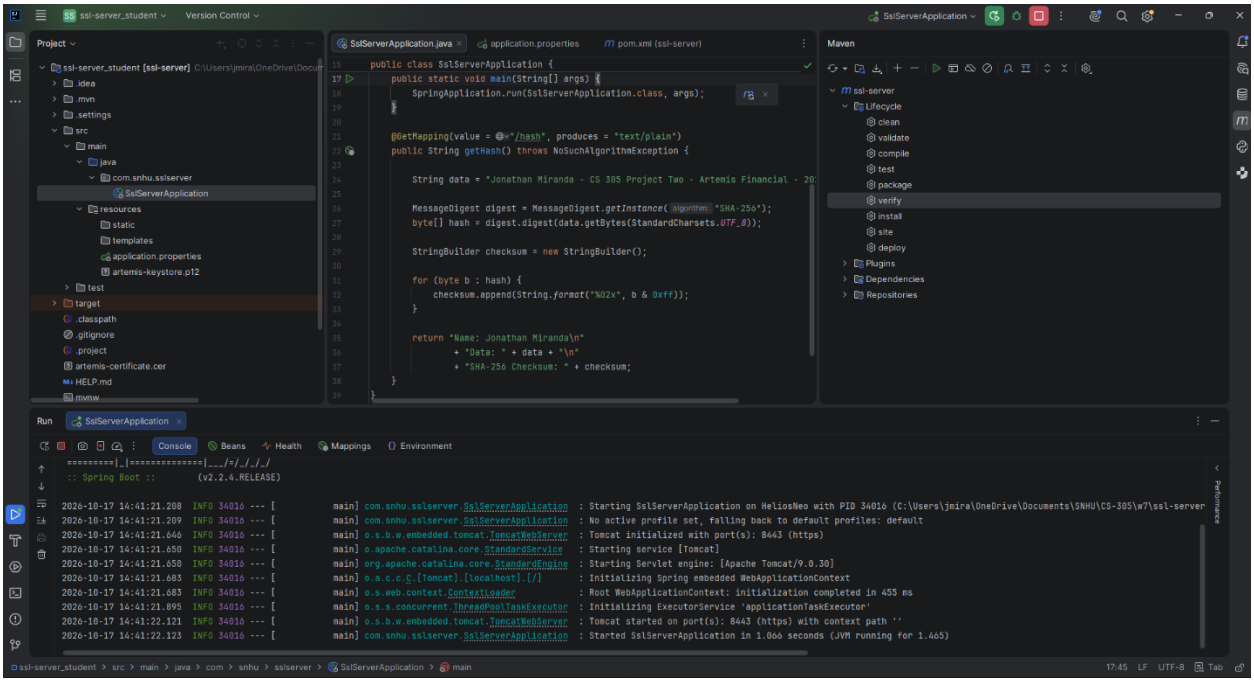


5. Secondary Testing





6. Functional Testing



```
==========================================
CS 305 PROJECT TWO - FUNCTIONAL TEST
==========================================

Application: Artemis Financial
Endpoint: https://localhost:8443/hash

HTTPS Enabled: True
HTTP Status: 200
Data Present: True
SHA-256 Match: True

Expected: 69e8214bbe1873589b5b3478891e532d87f7cd39980ccbe90df5ea427e281119
Actual:   69e8214bbe1873589b5b3478891e532d87f7cd39980ccbe90df5ea427e281119

==========================================
FUNCTIONAL TEST: PASS
==========================================

PS C:\Users\jmira\OneDrive\Documents\SNHU\CS-305\w7\ssl-server_student>
```

7. Summary

This Vulnerability Assessment was conducted using the Vulnerability Assessment Process Flow Diagram provided by SNHU to review the Artemis Financial application against seven different security areas. Input Validation, APIs, Cryptography, Client/Server, Code Error, Code Quality

and Encapsulation were the seven areas identified in the diagram. For this project, the areas of greatest focus were Cryptography, APIs, Client/Server and Code Quality.

To accomplish the data integrity demonstration, I refactored the Java application to include a /hash endpoint to generate a SHA-256 checksum from a unique test string using Java's existing MessageDigest class. I also generated a self-signed certificate for use with the Java application, which now uses HTTPS on port 8443 instead of HTTP.

After refactoring the code for the new endpoint, I tested the /hash endpoint independently, first by using PowerShell to calculate the SHA-256 checksum of the original test data and then by comparing this against the output of the newly created Java endpoint. The output from the PowerShell test and the newly created Java endpoint were identical and confirmed that the new code was working as required. In addition to testing the new code for the /hash endpoint, I also verified that the application responded over HTTPS and returned an HTTP 200 OK status. I manually reviewed the code for syntax, logic and security issues and ran OWASP Dependency-Check to identify vulnerabilities in the application's dependencies.

The new security features were added to the application, but it still has several security problems in the older libraries it uses. There were 254 security vulnerability findings listed across 14 of the application's dependencies. These will have to be reviewed and addressed before the application can be used in production.

8. Industry Standard Best Practices

When refactoring the Artemis Financial application, I followed industry standard secure coding practices to add extra security without affecting the existing functionality of the application. To create a SHA-256 hash I used Java's built-in MessageDigest class instead of attempting to roll my own solution. Also I used UTF-8 encoding when generating checksums to ensure that the same data would consistently result in the same checksum. As an example for the new /hash endpoint I used a fixed test string instead of user input to avoid input validation issues with this example.

For secure communication between a web client and web server of the Artemis Financial application I implemented HTTPS using a self-signed certificate generated using Java's Keytool. The certificate and private key were stored in a password-protected PKCS12 keystore. The password for this keystore was stored in an environment variable. This environment variable is not included in the source code of the Artemis Financial application. After refactoring I compiled the application and ran a test case to verify that the updated application functions as expected. In addition, I ran OWASP Dependency-Check on the application's dependencies to detect any known security vulnerabilities.

Using industry standard secure coding practices in developing Artemis Financial helps safeguard customer financial information as well as customer information. This helps in developing applications that will protect customer information during transmission as well as help in identifying and preventing possible vulnerabilities before they affect production and result in a security incident.

Using secure coding practices in the refactoring of the Artemis Financial application does not only relate to secure coding itself but also to the secure management and updating of the libraries an application is made of. In the example shown here, 254 vulnerability findings across 14 dependencies were identified in the OWASP Dependency-Check report. These findings have to be reviewed and addressed by Artemis Financial before the newly refactored application can be used in production. On a long-term basis, regular security tests, updates of all libraries and secure management of all configurations will be necessary to reach the required level of security for all applications of the company.

References

National Institute of Standards and Technology. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). https://doi.org/10.6028/NIST.SP.800-38D

National Institute of Standards and Technology. (2015). *Secure hash standard (SHS)* (Federal Information Processing Standards Publication 180-4). https://doi.org/10.6028/NIST.FIPS.180-4

National Institute of Standards and Technology. (2023*). Advanced encryption standard (AES)* (Federal Information Processing Standards Publication 197). https://doi.org/10.6028/NIST.FIPS.197